Proyecto 1 - Entrega 1 - Grupo 6 - DPOO

Integrantes:

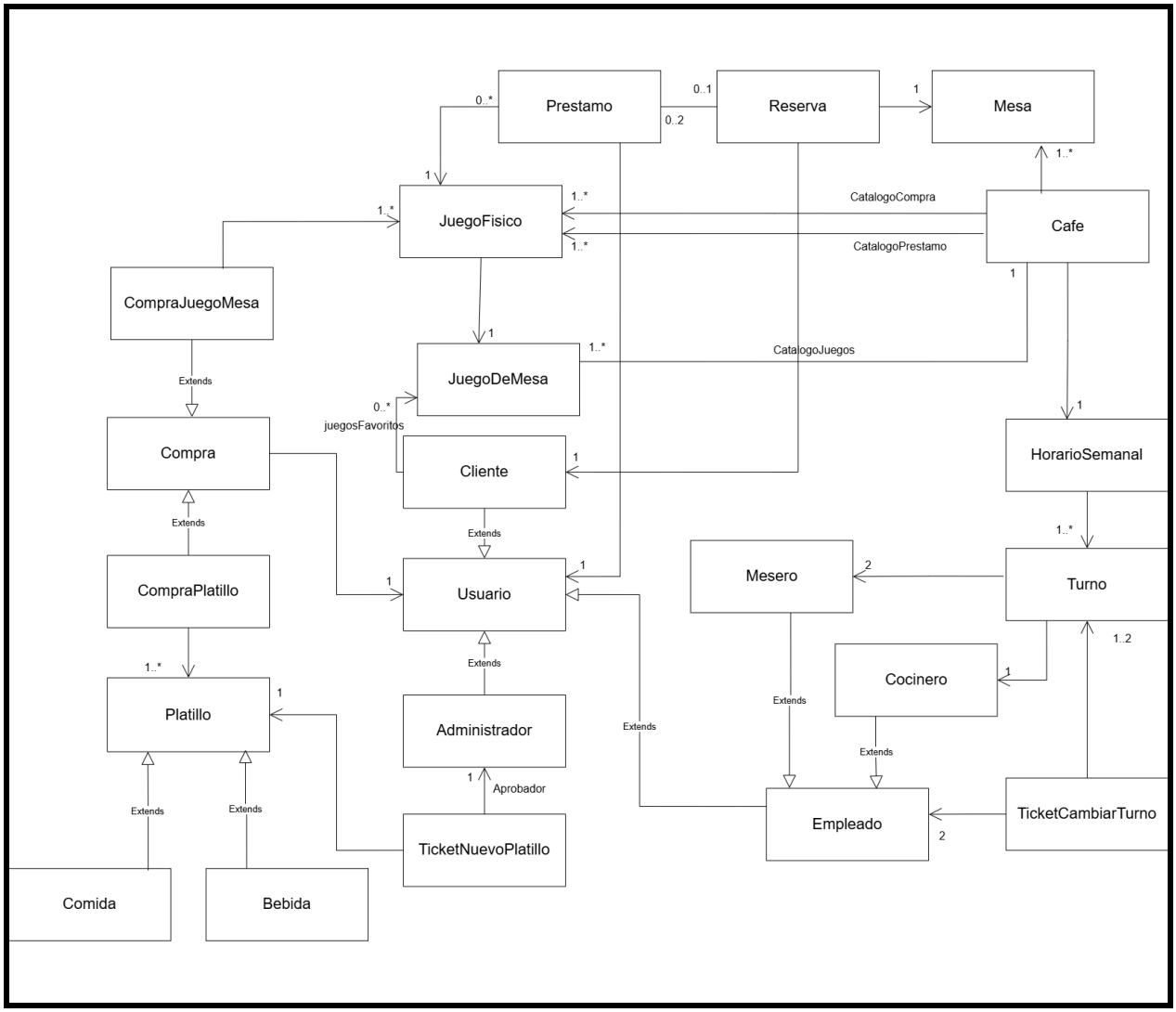
Carolina Ceballos Gómez - 202510267

Andrés Santiago Rodríguez Salazar - 202513997

Sebastián Felipe Angarita Rico - 202516784

Modelo de Dominio

Diagrama de clases:



Entidades identificadas y sus atributos

- **JuegoDeMesa:** representa un juego de mesa que está disponible en el café.

  Atributos:

  - ➔ nombre (String): nombre del juego de mesa
  - ➔ añoPublicacion (int): año en el que el juego de mesa fue publicado.
  - ➔ empresaMatriz (String): empresa que produjo el juego.
  - ➔ tipoJuego (String): representa la categoría/tipo de juego, puede ser Acción, Cartas o Tablero.
  - ➔ esDificil (boolean): indica si el juego es difícil, es decir si requiere o no una explicación previa de un mesero.
  - ➔ puedenNiños (boolean): indica si el juego es apto para niños, es decir, si puede ser entregado a una mesa con niños.
  - ➔ puedenJovenes (boolean): indica si el juego es apto para jóvenes, es decir si puede ser entregado a una mesa con jóvenes.
  - ➔ minJugadores (int): el número mínimo de jugadores para que el juego pueda ser entregado/utilizado.
  - ➔ maxJugadores (int): el número máximo de jugadores permitidos en el juego.
- **JuegoFisico:**

  Atributos:

  - ➔ estado (String): condición en la que se encuentra el juego, puede estar nuevo, bueno, falta una pieza, desaparecido, etc.
  - ➔ ocupado (boolean): indica si el juego está disponible o está prestado.
- **Usuario:** representa a una persona que puede acceder al sistema. Tiene 3 herencias: Cliente, Administrador y Empleado.

  Atributos:

  - ➔ login (String): identificador único para acceder al sistema.
  - ➔ password (String): contraseña asociada al login.
- **Cliente:** representa un cliente que puede hacer compras, reservas, préstamos, etc

  Atributos:

  - ➔ puntosFidelidad (int): puntos acumulados por hacer compras en el café.
- **Empleado:** representa un trabajador del café. Se dividen en cocineros y meseros.

Atributos: no tiene atributos únicos.

- **Mesero:** desempeña el rol de atender mesas y explicar juegos.

  Atributos:

  - ➔ DificilesConocidos (List<String>): es la lista de los juegos difíciles que el mesero conoce y puede explicar.
- **Cocinero:** representa a la persona encargada de cocinar los platos y productos del menú.

  Atributos: no tiene atributos únicos.
- **Reserva:** representa la ocupación de una mesa por un grupo de clientes.

  Atributos:

  - ➔ numPersonas (int): cantidad de personas en la mesa.
  - ➔ hayMenores (boolean): indica si hay menores de edad en la mesa.
  - ➔ hayNinos (boolean): indica si hay niños menores de cierta edad en la mesa.
  - ➔ terminada (boolean): indica si la reserva ya finalizó.
  - ➔ mesaId (int): identificador único de la mesa.
  - ➔ prestamosActivos (Set<Prestamo>): lista de prestamos de la mesa.
- **Prestamo:**

  Atributos:

  - ➔ fecha (Date): fecha en la que se realizó el préstamo.
  - ➔ terminado (boolean): indica si el préstamo ya finalizó, o sea, si ya devolvieron el juego.
- **Mesa:** representa una mesa del café que puede ser ocupada por clientes.

  Atributos:

  - ➔ capacidad (int): número máximo de personas que pueden ocupar la mesa.
- **Compra:** representa la transacción realizada por un usuario del café.

  Atributos:

  - ➔ descuento (double): valor de descuento que se le aplicará a la compra.
  - ➔ total (double): valor total de la compra.
  - ➔ fechaHora (DateTime): fecha y hora de la compra.
- **CompraPlatillo:** representa la compra de un plato del menú de comida.

  Atributos: no tiene atributos únicos.
- **CompraJuegoMesa:** representa la compra de un juego de mesa.

Atributos: no tiene atributos únicos.

- **HorarioSemanal:** representa la distribución de los turnos en la semana.

  Atributos:

  - ➔ turnosPorDia (Map< (dia semana), Turno>): turnos que tienen los empleados según el día.

- **Turno:** representa el turno de trabajo asignado a cada empleado.

  Atributos:

  - ➔ diaSemana (String): turno del empleado en la semana.

- **TicketCambiarTurno:** representa la solicitud que puede hacer un empleado para cambiar su turno.

  Atributos:

  - ➔ empleadoPrincipal (Empleado): el empleado que realiza la solicitud.
  - ➔ turnoInicial (Turno): el turno que desea cambiar.
  - ➔ turnoFinal (Turno): el turno que desea obtener después del cambio o el nuevo asignado.
  - ➔ empleadoSecundario (Empleado): empleado con el que hará el cambio.
  - ➔ estado (String): estado actual de la solicitud, puede ser rechazada, aprobada o en revisión.

- **Platillo:** representa el producto del menú de comidas.

  Atributos:

  - ➔ precio (double): costo del platillo.
  - ➔ aprobado (boolean): indica si el platillo fue o no aprobado para aparecer en el menú.

- **Bebida:** representa una bebida disponible en el menú.

  Atributos:

  - ➔ esAlcoholica (boolean): indica si la bebida es alcohólica, es decir, si se puede dar o no en una mesa con menores de edad.
  - ➔ esCaliente (boolean): indica si la bebida es caliente o no, es decir, si se puede dar o no en una mesa con un préstamo de juego tipo acción.

- **Comida:** representa un producto de la pastelería.

  Atributos:

- ➔ alergenos (List<String>): lista de posibles alérgenos en el producto.

- **TicketNuevoPlatillo:** representa la solicitud de agregar un nuevo plato al menú.
  Atributos:
  aprobado (boolean): indica si la solicitud para agregar el nuevo platillo fue aprobada o no.
- **Café:** representa el establecimiento en donde se ofrecen los juegos y el menú.
  Atributos:
  - ➔ capacidadClientes (int): número máximo de clientes que pueden estar en el local al mismo tiempo.
  - ➔ catalogoCompra (List<JuegoFisico>): lista de juegos físicos disponibles para la venta.
  - ➔ catalogoPrestamo (List<JuegoFisico>): lista de juegos físicos disponibles para préstamo.
  - ➔ menu (List<Platillo>): lista de platillos aprobados disponibles para la venta en el café.
  - ➔ prestamos (List<Prestamo>): registro de los préstamos de juegos realizados en el café.
- **Administrador:** representa la persona que puede organizar y aprobar turnos semanales, que gestiona el inventario y que también supervisa el menú y sus cambios.
  Atributos: no tiene atributos únicos.

Relaciones entre las entidades

Herencia:

(Padre ← Hijo).

Herencias de Usuarios:

- Usuario ← Cliente.
- Usuario ← Empleado.
- Empleado ← Mesero.
- Empleado ← Cocinero.
- Usuario ← Administrador.

El propósito principal de la clase Usuario es manejar todo el sistema de login. Adicionalmente, la clase es referenciada cuando se quiere ejecutar una compra o un préstamo. Estas características se transfieren a todos sus hijos.

Herencia de Platillos:

- Platillo ← Bebida.
- Platillo ← Comida.

La principal razón detrás de la separación entre las comidas y las bebidas es que ambas manejan funcionamientos diferentes; las bebidas tienen las restricciones de no poder ser vendidas si hay menores de edad o se están jugando juegos de acción, dependiendo de los atributos de la bebida. Por otro lado, las comidas tienen que tener advertencias sobre posibles alérgenos. De cualquier manera, necesitamos poderlas tratar de manera similar para cuando los clientes quieran ordenar algún platillo.

Herencia de Compras:

- Compra ← CompraJuegoMesa.
- Compra ← CompraPlatillo.

La razón de no usar genericidad en este caso es debido a que se manejan diferentes tipos de comprobaciones de validez para cada clase; por ejemplo, para poder vender un juego de mesa necesitamos que haga parte del catálogo de venta, mientras que tenemos que hacer las comprobaciones de alcohol con menores, etc., para los platillos. La alternativa sería crear una clase *producto,* pero generaría la misma cantidad de conexiones en el diagrama, por lo que consideramos esta solución más intuitiva.

Relaciones del sistema de turnos.

- Cafe → HorarioSemanal: Es intuitivo que el café tiene que tener empleados; por consecuencia, necesita que tenga un sistema de turnos controlado por el horario semanal, por eso tenemos una referencia de 1 obligatoria.
- HorarioSemanal → Turno: Esta conexión representa que un horario gestiona los turnos de todos los días de la semana, dado que no sabemos cuántos días a la semana funciona el café, o cuáles, no podemos hacer referencias específicas. En cambio, sabemos que un café que no funcione no tiene sentido, en el mundo lícito, así que sería una relación obligatoria.

- Turno → Mesero, Turno → Cocinero: Como lo menciona el caso de estudio, hay exactamente un cocinero y dos meseros por turno. La clase Turno es tan solo una manera de estructurar la información de quién está en turno para un punto dado de la semana.
- TicketCambiarTurno → (Turno, Empleado): El ticket de turno, permite hacer dos cosas, intercambiar a dos personas de sus turnos, este es el caso para el que necesitamos 2 empleados y 2 turnos. En el caso de que contratasemos un nuevo empleado y le quitesemos un turno a un empleado actual, en este caso la segunda referencia del turno sería irrelevante. Algo importante es que para cualquier caso los empleados deben ser del mismo tipo, pues no podemos tener a un mesero como cocinero o viceversa.

Relaciones del sistema de restaurante:

- TicketPlatilloNuevo → Platillo: Un ticket tiene que hacer referencia a un platillo aún por aprobar o no. Esto nos permitirá tener trazabilidad de los platillos agregados, de cualquier manera, es intuitivo que es necesario tener un platillo para el ticket. Este tiquete guardará la referencia del administrador, quien aprobó la solicitud.
- Café → Platillo: Dado que nuestra clase café representa la mayoría de cosas que le pertenecen al café, en las que está incluido el menú (lo que representa esta relación), este *menú* hace referencia también a los platillos que todavía no han sido aprobados, cuando se imprima este menú, se van a obviar los platillos no aprobados.

Relaciones del sistema de Juegos:

- JuegoFisico → JuegoDeMesa: Dado que queríamos evitar repetir información de los juegos, optamos por tener instancias de los juegos de mesa, llamados juegosFisicos, que son los productos que se pueden vender, comprar y prestar; no tiene sentido tener un juegoFisico que no esté asociado a un juego de mesa.
- Cliente → JuegoDeMesa: Como menciona el caso de estudio, debemos guardar los juegos favoritos de los clientes, una persona podría no tener juegos favoritos o múltiples.
- Café → JuegoDeMesa: Como se ha establecido, el café guarda las referencias de la mayoría de cosas que necesitan acceso constante en la aplicación, incluyendo los juegos que ya hemos inicializado, para que en el caso de agregar un juego Físico nuevo no haya que crear otra vez toda su información, y sepamos qué juegos tenemos.

- Cafe → JuegoFisico: Dado que hay que tener seguimiento del stock de venta y prestamo de juegos fisicos, guardamos referencias a todos los juegos que tenemos en alguno de estos catalogos.
- Prestamo → (JuegoFisico, Usuario, Reserva): El núcleo de este sistema, el prestamo necesita un par de referencias claves, un juegoFisico obligatorio, pues no tendría sentido hacer un préstamo de nada, el usuario que quiere hacer el préstamo, por la misma razón, es obligatorio. La razón por la que la reserva es opcional es pues, en el caso de que el usuario sea un empleado, no sería necesario.
- Reserva → (Mesa, Cliente, Prestamo), Cafe → Mesa:
  Es importante aclarar que una mesa es un objeto que representa un lugar donde se puede alojar una reserva; el café tiene que guardar una referencia de todas las mesas, para saber cuáles están disponibles a la hora de crear una reserva. Las reservas tienen las anteriormente mencionadas mesas. Adicionalmente, las reservas cuentan con un único cliente obligatorio, pues no tendría sentido lo contrario. Por último, las reservas tienen un de préstamos, de 2, pero este puede ser opcional, pues tiene sentido que una persona primero tenga su mesa y después preste juego o que directamente no lo haga.

Relaciones del sistema de compra:

- Compra → Usuario: La única razón por la que contamos con esta intuitiva asociación es porque la compra solo se encarga de la lógica del pago (la abstracción que tenemos que hacer de este concepto) y de guardar registro de cuándo pasó.
- CompraPlatillo → Platillo, CompraJuegoMesa → JuegoFisico:
  Es intuitivo que para todas las compras tenemos que tener algún objeto que estamos comprando; la razón de usar diferentes clases es porque cada una maneja los impuestos de manera diferente. Esta fue la manera más simple de representar este comportamiento; además, nos permite filtrar por tipo de venta, útil para el panel de control del administrador. Es intuitivo que una compra tiene que tener al menos un producto.

Reglas del dominio (ajustadas al modelo)

- La cardinalidad de la relación Reserva -> Prestamo es (0..2). Es decir que una reserva puede separar máximo 2 juegos simultáneamente.
- Todas las reservas tienen un id de mesa válido.

- La suma de las personas de una nueva Reserva y el total de personas de las reservas activas no puede exceder la capacidadClientes del Café.
- Cuando se genere el subcatálogo para una reserva, todos los elementos en este deben tener la cantidad de personas en la reserva dentro de su rango.
- En el subcatálogo generado para una reserva, solo deben haber juegos que cumplan con las restricciones de niños y jóvenes.
- Un juego válido no puede tener el caso puedenNiños = true, puedenJovenes = false.
- Una reserva no puede tener hayMenores=false, hayNinos=true.
- Los juegos que estén en el subcatálogo aparecerán como disponibles si existe un juegoFisico en catalogoPrestamo con ocupado=false.
- No se puede crear un préstamo con un juegoFisico que este marcado con un ocupado=true.
- Un empleado solo puede realizar un préstamo si no está asignado al turno activo del día.
- Si el usuario de un préstamo es un cliente, es obligatorio que tenga una reserva asociada.
- Un préstamo solo se puede realizar si hay al menos 1 copia del JuegoFisico disponible (ocupado=false).
- Cada turno debe tener al menos 1 cocinero y 2 meseros asignados. Y por ende no se puede aprobar un cambio de turno si deja el café con menos de 1 cocinero o 2 meseros.
- Las bebidas alcohólicas no pueden ser vendidas a mesas donde hayan menores de edad (hayMenores=true).
- Las bebidas calientes no pueden ser vendidas a reservas con préstamos de juego tipo Acción.
- Los juegos tipo Acción no pueden ser prestados a reservas con bebidas calientes.
- El descuento de empleados es del 20% para ellos y 10% para los clientes cuando lo comparten.
- Todas las compras de juegos deben incluir un IVA del 19% en su valor total.
- Todas las compras de comida deben incluir un impuesto al consumo del 8% en su valor total.
- El administrador no puede alquilar juegos ni hacer compras.
- Solo el administrador puede mover juegos entre los inventarios.
- Cada compra le genera un 1% de su valor en puntos de fidelidad al cliente.

- Cada punto de fidelidad equivale a $1.
- Los puntos de fidelidad solo pueden usarse como descuento en compras futuras.
- El estado de un juego (Nuevo, bueno, falta una pieza, etc) solo puede ser modificado por el administrador.
- Si no hay un mesero capacitado para un juego difícil, se puede prestar, pero con una advertencia.
- En las ventas, la propina sugerida es 10% antes de impuestos, pero el cliente puede cambiarla.

Requerimientos Funcionales

REQS del usuario:

- **RF1: Registro**

  Como usuario, quiero registrarme en el sistema con un login y una contraseña, para poder acceder a las funcionalidades del café.

  Criterios de aceptación:

☐ Crear y registrar un login y su contraseña

☐ Asegurarse de que el login sea único y no haya sido registrado antes

☐ Almacenar la contraseña de forma segura, garantizando la privacidad del usuario

- **RF2: Inicio de sesión**

  Como usuario, quiero iniciar sesión con mi login y contraseña, para acceder a mi cuenta en el sistema.

  Criterios de aceptación:

☐ Permitir el ingreso con login y contraseña

☐ Mantener la sesión activa hasta que el usuario la cierre.

☐ Validar los datos y asociarlos con el usuario al que le pertenecen (es cuenta única para cada usuario).

- **RF3: Cerrar sesión**

  Como usuario, quiero cerrar mi sesión para proteger mi cuenta una vez que termine de utilizar el sistema.

  Criterios de aceptación:

☐ Incluir el botón de cerrar sesión y al oprimirlo redirigir a la cuenta de inicio

- ☐ Cerrar la cuenta y solo dejar el ingreso de nuevo con login y password

- **RF4: Crear reserva**

  Como Cliente, quiero poder crear una reserva indicando número de personas, si hay niños, y si hay menores, para poder acceder a los juegos y al menú del café.

  Criterios de aceptación:

- ☐ Permitir ingresar el número de personas
- ☐ Permitir indicar si hay niños y menores
- ☐ Verificar que la reserva no exceda la capacidad máxima del café
- ☐ Asociar la reserva con una mesa con capacidad suficiente para el número de personas.

- **RF5: Ver catálogo de juegos**

  Como cliente, quiero ver el catálogo de juegos disponibles para préstamo según mi reserva, para poder escoger un juego adecuado y de mi gusto.

  Criterios de aceptación:

  - ☐ Mostrar juegos según el número de personas en la reserva
  - ☐ Mostrar juegos según la restricción de edad
  - ☐ Verificar restricciones de tipo de juego y tipo de bebida
  - ☐ Mostrar disponibilidad de cada juego físico.

- **RF6: Seleccionar juegos para préstamo**

  Como cliente, quiero seleccionar hasta dos juegos para préstamo, para poder jugar durante mi reserva.

  Criterios de aceptación:

  - ☐ Limitar la selección a un máximo de 2 juegos al tiempo por reserva.
  - ☐ Verificar disponibilidad de copias físicas.

- **RF7: Solicitar préstamo**

  Como cliente, quiero solicitar el préstamo de los juegos seleccionados, para utilizarlos en el café.

  Criterios de aceptación:

  - ☐ Verificar restricciones de edad, cantidad de personas y bebidaCaliente.
  - ☐ Asociar préstamo con la reserva correspondiente.
  - ☐ Registrar el préstamo en el sistema.

- ☐ Marcar los juegos físicos como ocupados

- **RF8: Devolver juegos**

  Como cliente, quiero devolver los juegos prestados, para poder estar a paz y salvo cuando termine de usarlos.

  Criterios de aceptación:

  - ☐ Volver a marcar el juego como disponible.
  - ☐ Registrar la devolución.
  - ☐ Revisar y actualizar el estado del juego que acaba de ser devuelto.

- **RF9: Finalizar reserva**

  Como cliente, quiero finalizar mi reserva para notificar que ya he acabado mi estancia en el café.

  Criterios de aceptación:

  - ☐ Modificar e indicar que la reserva ya terminó.
  - ☐ Asignar esa mesa como disponible.
  - ☐ Verificar que los pagos hayan sido realizados y los préstamos devueltos.

- **RF10: Compra de juegos**

  Como cliente, quiero comprar juegos del catálogo de compra para adquirirlos y llevarlos a casa.

  Criterios de aceptación:

  - ☐ Mostrar catálogo de juegos para la compra disponibles.
  - ☐ Permitir seleccionar 1 o más juegos para la compra.
  - ☐ Calcular el total de la venta según juegos, cantidad de ellos e incluir impuestos.
  - ☐ Registrar la compra en el sistema.

- **RF11: Compra de platillos**

  Como cliente quiero comprar productos del menú para consumir alimentos o bebidas durante mi reserva.

  Criterios de aceptación:

  - ☐ Mostrar menú de platillos aprobados y disponibles.
  - ☐ Permitir seleccionar 1 o varios platillos.
  - ☐ Revisar restricciones de bebidas de edad o con los juegos de mesa de la reserva.

- ☐ Calcular el total de todos los platillos e incluir impuestos.
- ☐ Registrar la compra en el sistema.

- **RF12: Usar puntos de fidelidad**

  Como cliente quiero utilizar los puntos de fidelidad que he acumulado durante mis compras para obtener descuentos.

  Criterios de aceptación:

  - ☐ Mostrar el total de puntos de fidelidad y el saldo al que equivale.
  - ☐ Aplicar descuento correspondiente a la compra.
  - ☐ Actualizar los puntos y el saldo después de ser utilizados en una compra.

REQS del empleado:

- **RF13: Consultar turno semanal**

  Como empleado quiero poder visualizar mis turnos de la semana, para saber qué días debo ir a trabajar.

  Criterios de aceptación:

  - ☐ Mostrar lista de turnos asignados al empleado.
  - ☐ Indicar para cada turno el día y la hora de inicio y finalización.

- **RF14: Solicitar cambio de turno**

  Como empleado quiero solicitar un cambio de turno para modificar mi horario y acomodar mis compromisos externos al trabajo.

  Criterios de aceptación:

  - ☐ Permitir seleccionar el turno actual y el nuevo turno deseado, incluyendo al empleado con el que se hará el cambio.
  - ☐ Registrar solicitud en el sistema.
  - ☐ Actualizar el estado de la solicitud (pendiente, aprobada, rechazada).

- **RF15: Sugerir nuevo platillo**

  Como empleado quiero poder sugerir un nuevo plato para el menú, para así poder expandir nuestra oferta gastronómica.

  Criterios de aceptación:

  - ☐ Permitir ingresar el nombre y la descripción del nuevo platillo, adjuntando la receta y la lista de alérgenos.

- ☐ Proponer un precio tentativo para el platillo.
- ☐ Registrar la solicitud y asignar estado como “pendiente de aprobación”.

- **RF16: Compra con descuento de empleado**

  Como empleado quiero comprar productos del menú o juegos con mi descuento de 20%, para así ahorrar y recibir mis beneficios por mi rol en el café.

  Criterios de aceptación:

  - ☐ Aplicar el 20% de descuento al total de la compra.
  - ☐ Registrar la compra con el descuento aplicado.
- **RF17: Compartir descuento con clientes**

  Como empleado quiero compartir mi descuento con los clientes, para que a ellos se les pueda aplicar un 10% de descuento.

  Criterios de aceptación:

  - ☐ Compartir un código de descuento solo para esa compra.
  - ☐ Permitir que el cliente ingrese el código y lo utilice.
  - ☐ Aplicar el 10% de descuento al total del pago del cliente.
  - ☐ Registrar la compra con el descuento aplicado.

REQS del mesero:

- **RF18: Explicar un juego a clientes**

  Como mesero quiero explicarle el juego difícil que conozco al cliente que lo haya solicitado, para que pueda comprender las reglas y la dinámica y así jugarlo sin confusiones.

  Criterios de aceptación:

  - ☐ Verificar que el mesero conozca el juego difícil pedido en la mesa.
  - ☐ Registrar la ayuda y si no hay ningún mesero que sepa ese juego, dar la advertencia de que tal vez no entiendan cómo jugarlo.

REQS del administrador:

- **RF19: Modificar catálogos**

  Como administrador quiero ver y modificar todos los catálogos del café (alquiler, compra y menú) para mantener y actualizar la información del sistema.

Criterios de aceptación:

- ☐ Visualizar la lista de los juegos físicos disponibles en cada catálogo y los platillos disponibles en el menú.
- ☐ Agregar nuevos juegos a los catálogos o nuevos platillos al menú.
- ☐ Editar o modificar info de juegos/platillos ya existentes en los catálogos.
- ☐ Eliminar juegos o platillos de los catálogos si se requiere.

- **RF20: Resolver sugerencias**

  Como administrador quiero aprobar o rechazar sugerencias de nuevos platillos para el menú para decidir qué opciones si son rentables tenerlas y ofrecerlas.

  Criterios de aceptación:

  - ☐ Visualizar las sugerencias de platillos pendientes.
  - ☐ Revisar nombre, precio, descripción, etc.
  - ☐ Aprobar o rechazar la sugerencia al criterio del administrador.
    En caso de aprobación, agregar el platillo al menú.

- **RF21: Resolver cambios de turno**

  Como administrador quiero aprobar o rechazar las solicitudes de cambios de turnos de los empleados, para así gestionar y concretar el horario del personal.

  Criterios de aceptación:

  - ☐ Visualizar todas las solicitudes de turnos.
  - ☐ Revisar el turno que tiene actualmente y el que solicitó.
  - ☐ Revisar la restricción de que debe haber al menos 2 meseros y un cocinero en ese turno.
  - ☐ Aprobar o rechazar la solicitud.
  - ☐ En caso de aprobación, actualizar los horarios de los empleados afectados.

- **RF22: Mover juegos entre catálogos**

  Como administrador, quiero mover juegos del catálogo de compra al de préstamo y al revés, para ajustar la disponibilidad según la demanda.

  Criterios de aceptación:

  - ☐ Seleccionar un juego físico y cambiar su ubicación entre catálogos.
  - ☐ Al mover el juego, este debe desaparecer de un catálogo y aparecer en el otro.

- **RF23: Reparar juegos**

  Como administrador, quiero reparar un juego dañado reemplazando una copia del catálogo de préstamo por una de compra, para mantener correctamente el inventario de préstamo.

  Criterios de aceptación:

  - ☐ Seleccionar una copia dañada del inventario de préstamo.
  - ☐ Seleccionar una copia nueva del inventario de compra para reemplazarla.
  - ☐ La copia dañada se elimina o se marca fuera de servicio.
  - ☐ La copia nueva se mueve al inventario de préstamo.
- **RF24: Marcar juego como desaparecido**

  Como administrador, quiero marcar un juego no devuelto como desaparecido, para actualizar el catálogo y tomar acciones si es necesario.

  Criterios de aceptación:

  - ☐ Seleccionar un préstamo vencido y marcar el juego como desaparecido.
  - ☐ El juego se elimina del catálogo de préstamo y se marca como no disponible.

Restricciones del Proyecto

- **Lenguaje de programación:** El sistema debe ser implementado en Java debido a lo que nos pide el curso sobre diseño y programación orientada a objetos. Además, esto garantiza el uso de un solo lenguaje para mantener unificado el código y alinearse con los conocimientos de la clase.
- **Persistencia en archivos:** Toda la información como usuarios, juegos, reservas, préstamos, compras, etc, será persistida en archivos planos o JSON, para que cuando el programa se cierre y se vuelva a usar, toda la información debe estar correctamente. Al no hacer uso de bases de datos, se mantiene la simplicidad del proyecto y permite enfocarse en la lógica de la aplicación.
- **Consistencia del formato:** El formato de los archivos debe ser consistente en toda la aplicación. Esto significa que si se elige JSON, toda la información debe persistir siguiendo una estructura uniforme para facilitar su lectura y escritura.

- **Interfaz de usuario:** La aplicación no es necesario que tenga interfaz gráfica, el usuario podrá interactuar con el programa a través de la consola, haciendo pruebas que demuestren el completo funcionamiento de la aplicación.
- **Tiempo de desarrollo:** El proyecto debe desarrollarse durante unas semanas asignadas, con dos entregas, análisis y entrega. Haciendo que el tiempo sea limitado y se necesite planificación del trabajo para cumplir con el proyecto.
- **Tamaño del equipo:** El grupo de trabajo para realizar el proyecto es de 3 personas, definiendo una dinámica de trabajo en donde habrá distribución de tareas y trabajo en equipo
- **Control del proyecto:** Todo el código, documentos, archivos y versiones del proyecto se almacenarán en Git en un repositorio GitHub, permitiendo el trabajo colaborativo y el seguimiento de cambios.
- **Sin programas externos:** El programa no usará sistemas de pago externos, APIs u otros sistemas. Todos los trámites y procesos son simulados dentro del sistema y no requieren procedimientos con el mundo real.
- **Sin correo electrónico:** No se requerirán correos electrónicos a los usuarios para registrarse, confirmar compras o notificaciones. Toda la comunicación con el usuario se realizará por la interfaz de consola.
- **Multiplataforma:** La aplicación debe poder ejecutarse en cualquier sistema operativo que tenga instalada una Máquina Virtual de Java (JVM), esto se logra al mantener todo el programa en Java.
- **Control de acceso por rol:** El sistema controlará el acceso a funcionalidades según el rol del usuario (cliente, empleado, administrador), ya que un cliente no puede acceder a funciones de un administrador y al revés.
- **Privacidad de los datos:** Los datos, como información de login, compras, juegos favoritos, etc, de los clientes no serán visibles para otros clientes. Solo el administrador tendrá acceso a esto.

Programas de Prueba

Prueba 1 (Para RF1):

Un programa que podría probar el sistema de registro de usuarios en el sistema funcionaría algo así: Crearía 2 usuarios con el mismo login, el primero no debería generar ningún error, aunque cuando se fuese a crear el segundo debería dar un error. Adicionalmente se podría ejecutar otro algoritmo que cree 2 usuarios (con ids diferentes) del mismo tipo (ej clientes) y que el programa se asegure de que cuando obtengamos las referencias que tiene sean 2 objetos diferentes, esto nos permitiría comprobar que el sistema funciona correctamente.

Prueba 2 (para RF2):

Para poder comprobar este RF, crear un usuario; después, con la contraseña y login con el que lo instanciamos, tras usar el método de autenticación, debería validarnos, pues fue con las credenciales que lo creamos. El token de sesión que nos dé este nos debería permitir ejecutar acciones que requieran del permiso y la referencia de nuestro usuario. Si alguna de estas operaciones fallase, implicaría que la prueba fue no exitosa. La mayoría de estas acciones requerirán un café para poderse ejecutar, por lo que inicializamos uno.

Prueba 3 (para RF3):

Asumiendo que iniciamos sesión (en consecuencia, el sistema está iniciado de sesión), podríamos ejecutar la Prueba 2 como primer paso de esta. Después ejecutaremos el mecanismo de cerrar sesión, lo que debería invalidar todos los tokens de sesión funcionales en el momento. Si usásemos el token de sesión, no nos debería dejar hacer ninguna actividad con el privilegio de ese usuario.

Prueba 4 (RF 4):

Ya que es necesario ser un cliente autenticado para hacer una reserva, ejecutaremos el test 2 específicamente instanciando un cliente. Después ejecutaremos un par de comprobaciones; la primera se aseguraría de que se puede crear una instancia válida de una reserva, que debería dejarnos poner todos los atributos para después poder comprobar con los getters que toda la información cuadre, aunque haremos la particularidad de que la cantidad de personas sea la de una mesa arbitraria dentro del café que instanciamos previamente. Después, trataremos de crear una reserva con el mismo usuario antes de terminar la anterior, lo que no debería ser posible.

Después intentaremos hacer una reserva con un usuario diferente, pero que tenga una cantidad de personas mayor a la mesa disponible con más personas, lo cual debería fallar. Continuaremos

creando pares de clientes y reservas hasta llenar todas las mesas menos una, ajustándonos al máximo de personas por mesa, si hubiese algún error durante este proceso, sería una señal de que el sistema no funciona correctamente.

Prueba 5 (RF5, 6 y 7):

Primero, inicializamos un empleado como en el test 2. Adicionalmente agregaremos un número arbitrario de juegos de mesa y juegos físicos a nuestro café, específicamente al catálogo de juegos para préstamo, nos aseguraremos de que uno de los juegos de mesa solo tenga un juego físico. Después, activaremos la función de crear préstamo, lo que debería darnos un conjunto equivalente al de los juegos de mesa en el sistema, después trataremos de reservar el juego con una sola instancia. Después crearemos un cliente con su reserva, como en el test 4, nos aseguraremos de que la cantidad de personas esté en el rango de todos los juegos de mesa y que no tenga menores o jóvenes. Tras esto trataremos de crear un préstamo con él. En el catálogo resultante nos deberían aparecer todos los juegos en el sistema, aunque uno debería estar marcado como ocupado, pues todas las instancias (una) están ocupadas por el empleado que creamos, elegiremos un juego arbitrario. Después trataremos de instanciar 2 préstamos más asociados al mismo cliente, y en consecuencia a la reserva. A la hora de hacer el tercer préstamo del cliente debería fallar, pues tenemos el límite de 2. Todo esto lo haremos con parámetros de los préstamos válidos, ahora intentaremos generar préstamos no válidos, crearemos nuevos clientes con sus reservas, donde seleccionaremos combinaciones (válidas e inválidas) de parámetros, y ejecutaremos todos (deberían fallar los que tengan problemas con las restricciones del dominio). Para esto necesitaremos generar compras asociadas a algunas reservas.

Prueba 6 (RF 8):

El test creará un usuario y préstamo válidos (se ejecutará para empleado y cliente). Entonces después, haremos que el administrador del sistema (que generamos junto al café) ejecute un método de terminarPrestamo (o parecido), para este será necesario que el token de autenticación del administrador. Tras ejecutar el método, todos los juegos deberían estar disponibles (porque el préstamo que creamos era el único existente) y también deberíamos ser capaces de crear 2 nuevos préstamos como el mismo usuario, ya que el otro estaría como terminado.

Prueba 7 (RF 9):

Podríamos ejecutar el test 4, tras esto crearíamos dos préstamos de dos juegos arbitrarios y trataríamos de terminar la reserva, esto nos daría un error, después terminaríamos el préstamo.

Posteriormente cancelaríamos un solo préstamo y también debería fallar, sería hasta que no haya ningún préstamo activo que se debería ejecutar sin inconvenientes. Podríamos hacer esto mismo con un número arbitrario de compras sin validar en caja, haciendo que un mesero las valide gradualmente hasta llegar a cero, todos los intentos de cerrar la reserva deberían fallar menos el último.

Prueba 8 (RF 10):

Para este test primero crearíamos un mesero autenticado (reutilizando el flujo del test 2) ya que este es el tipo de usuario que debería poder verificar las compras para el café. Después inicializaremos el café con un catálogo de juegos de mesa, asegurándonos de que algunos tengan varias instancias físicas y otros solo una, con todos los juegos físicos en el catálogo de venta. Después crearemos un cliente y un empleado arbitrario. Trataremos de crear una compra apuntando al cliente al juego que tiene un solo juego físico. Antes de confirmar el pago con el mesero, trataremos de comprar el mismo juego con el empleado, la operación debería fallar. Posteriormente autenticados como el mesero, confirmaremos la compra del cliente. Esto debería cambiar el estado del juego físico de “en venta” a “vendido”.

Prueba 9 (RF 11):

Para este test primero crearíamos un mesero autenticado (reutilizando el flujo del test 2) ya que este es el tipo de usuario que debería poder confirmar las compras de platillos dentro del café. Después inicializaremos el café con un catálogo de comida y bebidas que incluya algunos platillos normales y al menos una bebida alcohólica y una caliente. Posteriormente crearemos un cliente autenticado con una reserva válida (reutilizando parte del flujo del test 4).

Primero haremos un caso sencillo donde el cliente compre un platillo cualquiera del catálogo. La compra debería crearse correctamente y, cuando el mesero autenticado confirme el pedido, este debería quedar registrado como una compra válida asociada al cliente.

Después probaremos algunos casos que deberían fallar. Hagamos un caso en el que el cliente tenga un préstamo activo de un juego de acción (reutilizando parte del flujo del test 5). Si ese cliente intenta comprar la bebida caliente que creamos mientras tiene ese préstamo activo, la operación debería fallar y el sistema no debería permitir registrar la compra, cuando terminemos la compra, debería ser exitosa.

Adicionalmente haremos otro caso en el que el cliente tenga menores asociados a su reserva e intente comprar una bebida alcohólica del catálogo. En ese caso el sistema debería rechazar la

operación, ya que no debería ser posible comprar bebidas alcohólicas cuando hay menores en la mesa. Adicionalmente, podríamos crear una compra en la que haya algunos productos válidos y otros no, para comprobar que el sistema funciona correctamente.

Prueba 10 (RF 12):

Podríamos ejecutar el test 8 o 9 para tener un cliente válido con compras y en consecuencia puntos de fidelidad. Después trataríamos de crear otra compra válida de un producto (por ejemplo un juego con varias copias) y cuando fuésemos hacer la compra, deberíamos seleccionar la opción de "aplicar puntos" que debería tener la interfaz, esto nos daría una compra con un valor total "x", después generamos otra compra (es irrelevante si se hace con el mismo usuario) al mismo juego, diferente juego físico obviamente, deliberadamente desactivando la opción de usar puntos, esto nos daría un valor "y". Deberíamos asegurarnos de que "x < y". Si no, el test falla.

Prueba 11 (RF 13, 14):

Para este test generamos un caso en el que la cafetería trabaje por lo menos 3 días a la semana, y que cada día tenga un turno válido, entonces tendríamos que tener al menos 1 cocinero y 2 meseros. Adicionalmente, digamos que el mesero C trabaja los miércoles, A trabaja de lunes a miércoles y B de lunes a martes. Si nos autenticásemos como cualquiera de los 4 empleados de nuestro sistema, veríamos el horario completo, donde nos debería decir quiénes están en cada rol todos los días. Así (conectando con el RF 14) podríamos seleccionar en el menú la opción de crear un ticket, tengamos un horario asignado en este momento o no. Este nuevo tiquete lo podríamos asignar para que C trabaje los lunes y B los miércoles; esto debería poder hacerlo cualquiera de los 2, asumamos que lo hace C. Entonces C seleccionaría el turno del lunes y a B, después, su turno del miércoles, finalmente enviaría el ticket. Después podríamos, como el administrador del café, aceptar el ticket. El horario debería mostrar el cambio que reflejamos, si no lo marcaríamos como no exitoso. Cuando se mostrase el menú ía aparecer que la jornada de todos los días es única, de XX-XX a XX-XX, así los empleados sabrían cuando tienen que ir a trabajar.

Prueba 12 (RF 15):

Necesitaríamos crear un empleado arbitrario válido y autenticarnos como tal, adicionalmente, la autenticación del administrador también sería válida para esta feature. Después podríamos seleccionar la opción de proponer platillo, esto abrirá un menú donde agregaremos todos los

parámetros, si es bebida o comida, precio, alérgenos (en caso de ser comida) o fría, alcohólica, caliente (en caso de ser bebida) etc. Después de confirmar el ticket, debería haberse creado el platillo con la etiqueta “aprobado” como falsa, si no activaríamos una excepción. Posteriormente el administrador podría entrar y aprobar alguna de las comidas propuestas. Esto debería hacer que el platillo se marque como aprobado y que el platillo aparezca en el menú.

Prueba 13 (RF 16):

Después de generar un empleado válido y un producto (haríamos esto para comida y juegos). Cuando tratemos de crear la compra con el usuario siendo este empleado y confirmemos la compra, la compra debería tener como atributo de descuento un valor positivo equivalente al 20% del valor total.

Prueba 14 (RF 17):

Asumamos que tenemos un empleado y un cliente, el cliente va a hacer una compra, y en uno de los parámetros de la view de la acción le aparece una manera en la que puede introducir el login (sin contraseña) de dicho empleado, si este es válido, cuando se genere la compra (si es que es válida), el valor de descuento será el 10% del total. Una manera no redundante de probar este comportamiento sin probar otros RFs, es tratar de hacer la cuenta y comprobar la relación entre el descuento y el valor total.

Prueba 15 (RF 18):

Para probar este requerimiento, inicializamos un café con un menú y varios empleados (meseros y cocineros). Luego cada empleado crea sugerencias de nuevos platillos con nombre, información de platillo y su precio. Verificamos que cada sugerencia quede registrada en el sistema con un estado inicial de pendiente y que no aparecen en el menú del café. Comprobamos que los empleados puedan ver sus sugerencias y el estado de estas. Finalmente, intentamos que un cliente (no empleado) cree una sugerencia, lo cual debería fallar ya que un cliente no puede sugerir platillos.

Prueba 16 (RF19):

Para esta prueba, inicializamos un café con catálogos de compra, préstamo y menú con algunos elementos iniciales. Luego iniciamos sesión como administrador, y realizaríamos varias operaciones de modificación como: agregar un nuevo juego al catálogo de compra con varias copias nuevas, luego editamos información de un juego, como su precio o cambiar su estado, se eliminaría una copia del catálogo de compra y se agregaría un nuevo platillo al menú. Si se

intentase hacer esto desde un cliente o mover un elemento del menú a un catálogo, sería rechazado por falta de permisos o inconsistencia. Finalmente, verificamos que todos los cambios del catálogo se hayan realizado de manera exitosa.

Prueba 17 (RF20):

Primero inicializamos un café con un menú y un administrador. Luego, varios empleados (meseros y cocineros) crearán sugerencias de platillos, con su nombre y precio. Se verifica que estas nuevas opciones no salen en el menú todavía. Luego, el administrador inicia sesión y ve las sugerencias pendientes, aprobando las que considere, mirando si tienen alérgenos o si son bebidas alcohólicas, para luego aprobar algunas. Comprobamos que los platillos aprobados aparezcan ahora en el menú y puedan ser comprados por clientes, mientras que los rechazados no aparezcan ni puedan ser comprados. Finalmente, verificamos que los empleados puedan ver el estado de su sugerencia.

Prueba 18 (RF21):

Para esta prueba, configuramos un café con un horario semanal que incluya turnos específicos para sus empleados. Luego creamos varias solicitudes de cambio, unas con intercambio y otras que no. Las que dejen un día sin cocinero sin intercambio o 1 solo mesero deberían ser rechazadas por el sistema. Luego, el administrador revisa las solicitudes pendientes y aprueba el cambio a las solicitudes que dejen el café con 2 meseros y cocinero. Verificamos que los turnos se actualizan correctamente con las aprobaciones y que las rechazadas no modifican los turnos.

Prueba 19 (RF22):

Inicializamos un café con catálogos de compra y préstamo que tengan diferentes copias de juegos entre ellos. Luego, el administrador moverá una copia de un juego de un catálogo a otro, verificando que el número de copias de ese juego en el catálogo anterior se modificó y que ahora la nueva copia está en el nuevo. Para casos de error, se intentaría mover un juego que no pertenece a un catálogo a otro, mover un juego actualmente prestado, o que un cliente intente mover un juego, lo que debería dar error en todos los casos. Finalmente, se verifica que los cambios se hayan hecho de manera exitosa.

Prueba 20 (RF23):

Para esta prueba, inicializamos un café con juegos en ambos catálogos donde en el de préstamo tenemos una copia de un juego que tiene de estado “falta una pieza”, mientras que en el de compra hay dos copias de ese mismo juego en buen estado. Luego, como administrador,

haríamos la reparación seleccionando la copia del juego dañada en el catálogo de préstamo y la reemplazaríamos con una copia buena del catálogo de compra. Verificaremos que los movimientos se hicieron de manera exitosa y que la copia buena del catálogo de préstamo esté disponible para que un nuevo cliente la pueda usar.

Prueba 21 (RF24):

Inicializamos un café con un catálogo de préstamo que incluya varias copias de diferentes juegos de mesa, creamos varios préstamos activos, en donde varios clientes toman juegos por varios días. Pero uno de ellos no se devuelve después de un largo tiempo, por lo que el administrador revisa los préstamos pendientes y lo marca como desaparecido, al hacerlo, el préstamo se termina, su estado cambia a “desaparecido” y el juego se elimina del catálogo de préstamos. Se comprueba que esa copia del juego ya no aparezca en el catálogo y que tampoco pueda ser comprada.